

Humanoid 3v3 Soccer Using Unity ML-Agents

Jason Shao
261113937

tianzhen.shao@mail.mcgill.ca

Patrick Zakaria
261099252

patrick.zakaria@mail.mcgill.ca

Abstract—This project investigates the use of reinforcement learning to train humanoid agents to play 3v3 soccer in Unity using the ML-Agents framework. A two-stage curriculum was developed: first teaching stable locomotion with PPO, then transferring the learned model into a multi-agent soccer environment trained with MA-POCA and self-play. Agents used vector observations and continuous joint actions to walk, chase the ball, kick, and execute team-based strategies. The system showed consistent improvement in ELO and produced emergent behaviors such as coordinated kicking and reward-driven exploit strategies. While performance remains imperfect, the results demonstrate the effectiveness of curriculum learning and transfer learning for complex, physics-based multi-agent tasks.

Keywords—*Humanoid Control, Multi-Agent Reinforcement Learning, Unity ML-Agents, PPO, MA-POCA, Curriculum Learning, Self-Play, Transfer Learning, Soccer Simulation, Reward Shaping*

1) INTRODUCTION

Reinforcement learning applied to humanoid control remains a difficult challenge due to high-dimensional action spaces, unstable locomotion dynamics, and sparse rewards in complex environments. Inspired by prior work on game-playing AI systems and by projects from independent creators such as Code Bullet and AI Warehouse, we chose to build a visually engaging and entertaining experiment: training humanoid robots to play soccer in real time using Unity’s ML-Agents toolkit [1], [2].

The goal of this project is to develop a training pipeline that enables humanoid robots to walk, chase the ball, kick effectively, and coordinate with teammates. To tackle the complexity of this domain, we employ a two-stage curriculum-based transfer learning: solo locomotion learning with PPO, followed by multi-agent soccer training using MA-POCA and self-play. This approach allows the agents to transfer foundational motion skills into a more demanding, strategic environment [3].

2) BACKGROUND

This project is built using the Unity ML-Agents Toolkit, which provides a series of example environments and machine learning implementations that demonstrate locomotion, multi-agent interaction, and reward-driven learning. Two examples in particular informed our approach: the Walker humanoid model and the SoccerTwos soccer field environment. The Walker example offers a high-quality physics-based humanoid model with realistic joint articulation and proprioceptive observations, making it a suitable base for the humanoid movement. The SoccerTwos example provides a functional soccer field setup, with ball mechanics and a multi-agent team structure that can be repurposed for competitive gameplay [4].

In this project, we reuse the core assets from these examples which include the Walker humanoid prefab and the soccer field environment. However, because our objective differs substantially from the original templates, most of the

machine learning components had to be modified or completely redesigned, which includes the observation space of the humanoids, the reward functions, training plans, and overarching learning architecture.

These changes were essential to support humanoid 3v3 soccer which is significantly more complex than either environment individually. The full details of these modifications are explained in the following sections.

3) AGENT SYSTEM DESIGN

Since the default Walker and SoccerTwos templates were not designed for high-dimensional multi-agent soccer gameplay, we developed a custom observation space, action space, and learning pipeline tailored specifically to humanoid locomotion and soccer behavior.

The goal of the agent design is to encode sufficient information for balance, navigation, ball interaction, and team-based decision-making, while ensuring stable training in high-dimensional continuous control. This section describes the core structure of the agent’s perception and motor control mechanisms, which form the foundation for the reward functions and training strategy described later.

a) Observation Method

Our agent’s system adopts pure vector observations for both locomotion and soccer tasks. Although the ML-Agents package offers alternative perception methods using visual observations and raycast-based observations to offer richer spatial structure [5], they introduce architectural and training constraints that make them incompatible with a two-stage Walker Soccer workflow.

A humanoid requires precise joint positions, velocities, and body orientation cues to balance itself and imitate the actions of real soccer players. This information cannot be reliably inferred from first person images or raycasts. Hence, without vectorized proprioception, the agent would fail to maintain balance or learn a stable gait.

Furthermore, training performance plays a major role. Since visual policies take 5-20× longer to train and raycast policies take on average 2-4× longer compared to vector-only policies [5]. Given the large number of agents (six walkers) and the long training horizon for complex multi-agent soccer, visual or raycast-based methods were not feasible in the time allotted for this project.

Due to all the reasons stated above, vector observation remained as the best feasible option for the observation method for the humanoid agent.

b) Observation Space

Each agent receives a 269-dimensional observation vector, composed of two major components: a 243-element locomotion core provided by the example environment [4] and a custom 26-element soccer context module. The following is a summary of the elements of each core, but a full breakdown can be found in Appendix A.

(1) Locomotion Core (243 dimensions)

This component mirrors the proprioceptive structure used in advanced Walker models and provides detailed information about the humanoid’s biomechanics. It includes [4]:

- Relative positions, linear velocities, and angular velocities for all 16 body parts
- Local joint rotations and strength ratios for 13 controllable joints
- Global orientation cues such as hip-forward and head-forward quaternion deltas
- Velocity goal tracking features to promote directionally stable gait control
- A placeholder target position that is used during Stage 1 locomotion training and zeroed during Stage 2 soccer play

This representation captures all information needed for upright stability, omnidirectional movement, and balance recovery while ensuring that the policy remains invariant to global drift through the Orientation Cube reference frame.

(2) Soccer Context (26 dimensions)

To support tactical reasoning and role specialization, the agent receives a compact set of soccer-related observations:

- Ball position and velocity
- Team ID and assigned role (striker or goalie)
- Own goal and opponent goal positions
- Positions of up to two teammates and two opponents (for 2v2 or 3v3)

During Stage 1, all soccer inputs are zero-padded to maintain a fixed 269-dimensional structure, ensuring seamless weight transfer into Stage 2. In Stage 2, these observations become active and significantly enhance spatial awareness, enabling behaviors such as marking, clearing, and offensive positioning. The design intentionally limits the number of teammates/opponents observed to maintain dimensional efficiency while preserving essential tactical cues.

c) Action Space

The agent outputs a 40-dimensional continuous action vector, structured around torque-driven joint control and soccer interaction. Again, the following is a summary, but the full breakdown can be found in Appendix B:

- 26 joint rotation targets controlling the spine, chest, head, thighs, shins, feet, upper arms, forearms, and hands
- 13 joint strength scalars, enabling dynamic modulation of stiffness for improved responsiveness and ball handling
- 1 kick-intensity action, activated only after the curriculum unlocks ball-touch capability

This mapping provides highly flexible motor control, letting agents adjust posture stiffness, generate stable gaits, and coordinate limb trajectories for shooting or clearing the ball. Strength modulation is especially important for preventing rigid-gait local minima and allows adaptive behavior during physical contact with the ball or other agents. The kick action triggers a physics-based impulse when the ball is within range and curriculum conditions are met. Gating the kick through lesson progression prevents early-stage exploitative flailing and ensures agents first master locomotion before attempting coordinated ball interactions.

4) TRAINING METHODOLOGY

The training process for our humanoid soccer agents follows a structured two-stage curriculum-based transfer learning designed to gradually increase task complexity.

Instead of training the agents directly in a full soccer match, which would lead to unstable learning due to sparse rewards and challenging locomotion we separate the problem into (1) foundational locomotion learning and (2) multi-agent soccer learning.

Stage 1 focuses on developing stable balance, directional walking, and basic navigation on a simple flat surface. Stage 2 transfers the learned locomotion policy into a fully featured 3v3 soccer arena, where agents learn ball-chasing, kicking, positional movement, and team-based coordination through self-play.

This section describes the design choices, training objectives, reward structures, and curriculum strategies used to accomplish both stages.

a) Stage 1: Locomotion Solo Training

Stage 1 serves as the foundation for all subsequent soccer behaviors. The goal is to teach the humanoid to stand, walk, and move reliably toward designated targets. Because stable locomotion is essential for soccer but extremely difficult to acquire in a multi-agent setting, this capability must be learned first in isolation.

(1) Training Environment

Stage 1 is trained on a simple flat surface, as shown in the picture below. The environment includes a single humanoid agent on a flat, unobstructed floor. A green cube is randomly spawned within the field and represents a target the agent should walk toward. When the cube is touched or on every episode reset, the cube position is randomly respawned to encourage general-purpose locomotion, directional control, and turning. When the agent is detected to have fallen, the agent is respawned standing at the center of the arena in a random orientation. The random orientation also helps with generalization, encouraging the agent to learn to turn and adjust its orientation to find the target. Likewise, a random target walking speed is assigned per episode/respawn, to encourage the agent to learn to walk and “run”.

Overall, having this controlled environment removes unnecessary distractions and ensures the learning process is focused entirely on motor stability and navigation.

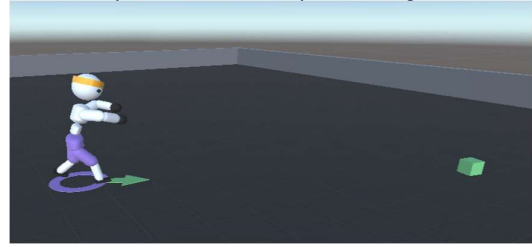


Figure 1 - Stage 1 Unity training environment

(2) Learning Algorithm

The Unity ML-Agents toolkit provides a multitude of example training algorithms to experiment with.

For Stage 1, we decided to use Proximal Policy Optimization (PPO) [3], a single-agent, on-policy RL algorithm, which we chose for its robustness in continuous-control tasks such as humanoid locomotion. In the case of locomotion, PPO prevents changing the policy too

drastically, ensuring that the agent learns walking without forgetting how to stand first.

The following explains how PPO was used in our case.

(3) Network Architecture

The network consists of a three-layer MLP with 512 units per layer, matching the architecture recommended for Walker-style humanoids in the Unity ML-Agents example environments documentation [1]. This capacity is sufficient to learn complex joint coordination patterns while remaining simple enough to limit training time.

(4) Experience Collection and Policy Updates

Over 1000 steps, the agent interacts with the environment, collecting experiences through the 269-observation vector. These experiences form tuples of the form (state, action, reward, next_state, done) which are then stored in a replay buffer.

As an on-policy algorithm, the PPO updates the policy using the collected batch with the following ratio clipped objective:

$$\text{ratio} = \frac{\pi_{\text{new}}(\text{action}|\text{state})}{\pi_{\text{old}}(\text{action}|\text{state})}$$

$$\text{clipped_ratio} = \text{clip}(\text{ratio}, 1 - \epsilon, 1 + \epsilon)$$

$$L_{\text{clip}} = \min(\text{ratio} * A, \text{clipped_ratio} * A)$$

With $\epsilon=0.2$, this policy ratio clipping prevents the policy from changing too drastically (more than 20%) in a single update. As mentioned before, this allows the agent to learn walking mechanics without forgetting basic skills like standing.

In the clipping, advantages A are computed using Generalized Advantage Estimation (GAE) with $\lambda=0.95$ and $\gamma=0.99$ as follows:

$$A_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l+1},$$

$$\text{where } \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

This smooths advantage estimates timesteps, helping the agent learn that:

- Walking forward consistently is valuable (high γ)
- Small corrections matter (high λ)

Additionally, we decided to add a $\beta=0.005$ entropy bonus to encourage exploration of different gaits and turning strategies to avoid premature convergence to suboptimal movement patterns.

(5) Architecture Preservation for Transfer Learnings

As mentioned before, during Stage 1, the agent receives the full 269-dimensional observation vector, but all 26 soccer-context values are zeroed. This preserves the network architecture and allows seamless weight transfer to Stage 2 without modifying inputs or retraining lower-level gait representations.

(6) Reward Structure

The reward structure in Stage 1 emphasizes motor stability and reliable movement toward the target cube. Rewards reinforce:

- Maintaining an upright posture
- Progress toward the target
- Stable locomotion and reduced falling
- Appropriate velocity alignment

By separating locomotion acquisition from soccer gameplay, the Stage 1 rewards remain simple, dense, and highly shaped, enabling fast convergence to a stable gait policy.

The value estimation for the state is computed from a mix of product and sum of the values from the following table. This is done rather than using a straight sum to encourage maximizing all components.

Table 1 - Stage 1 Rewards and Penalty Structure

Component	Value(s)	Purpose
Valid touch	+0.2 per target touch while standing	Reinforces upright, controlled gait rather than diving
Dive penalties	-0.15 to -1.35 based on angle from upright	Suppresses exploit of falling toward target for reset cycles
Turn encouragement	+0.05 $\times$ progress	Ensures omnidirectional capability (critical for soccer pursuit)
Standing penalty	-0.02 $\times$ angle frac	Discourages inertia lock when target moves laterally
Upright bonus	+0.03 per step	Maintains stable posture during locomotion training

(7) Training Configuration

The complete parameters for our training configuration can be found in the “WalkerSoccerStage1_Locomotion.yaml” file.

b) Stage 2: Soccer (Multi-Agent Training)

Stage 2 entirely focuses on teaching the agent soccer specific elements. The goal of this stage is to expand the previously learnt standing and walking model from Stage 1, to learn how to chase, kick and score as a team.

(1) Training Environment

The Stage 2 environment places the agents inside a full soccer field modeled after the SoccerTwos layout, now adapted for humanoid gameplay. Each episode features six humanoid walkers (three per team) along with a dynamic soccer ball using the Unity Physics engine. The field includes goal triggers, reset conditions, and team-dependent reward structures to support competitive multi-agent learning.

At the start of each episode, agents are positioned in fixed locations resembling simplified soccer formations (1 goalie and 2 strikers), ensuring consistent initial conditions while allowing diverse gameplay to emerge through self-play.

Additionally, in training mode, the ball spawns in randomized positions within the field center to further diversify gameplay.



Figure 2 - Stage 2 Unity training environment

(2) Curriculum-Based Lessons

A structured curriculum governs the transition from locomotion-focused behavior to competitive soccer play. Over the course of 3 lessons, curriculum parameters are

progressively modified to teach the agent to chase the ball, kick and score as a team.

This is done by enabling ball interaction in Lesson 1, enabling kicking in Lesson 2, and introducing self-play through ELO in Lesson 3 once locomotion is stable.

Table 2 - Curriculum Progressions Parameter Table

Parameter	Progression	Purpose
ball_touch	0.35 → 0.5 → 1.0 per touch per step	Gradually enables and scales ball interaction & kicking
ball_spawn_radius	1.5 → 2.0 → 1.5 from field center	Early chase central → broaden spatial search → tighten for scorinaf
locomotion_scale	0.5 → 0.4 → 0.15 per step upright	Fade locomotion shaping as soccer mastery increases
self_play_weight	0.0 → 1.0 @ 50% of total steps (per episode)	Automatic transition from cooperative skill acquisition to competitive play
locomotion_only	1.0 (Stage 1) / 0.0 (Stage 2)	Toggle soccer observation activation

(3) Learning Algorithm

For Stage 2, we decided to use MultiAgent Pothumous Credit Assignment (MA-POCA) [6], [7], a multi-agent cooperative, off-policy RL algorithm. This algorithm was chosen as it is specifically designed for cases requiring multi-agent cooperation and solves the credit assignment problem: “Which agent’s action led to the team goal?”

This was especially chosen over single-agent algorithms like PPO, since they struggle in multi-agent scenarios because they cannot distinguish individual contributions from team outcomes. In the case of MA-POCA, this is done through counterfactual reasoning as explained below.

(4) Network Architecture

The network reuses the same policy structure as Stage 1 with a three-layer MLP with 512 units per layer. However, a unique centralized critic is added to the architecture for team-wide value estimation:

Decentralized Actors (per agent):

- Input: 269-dimensional observation vector
- Output: 40 continuous actions
- Policy: Individual agent decides actions based on local observations

Centralized Critic (team-wide):

- Input: Concatenated observations from all teammates (269 x agents_per_team)
- Output: Single value estimation $V_{team}(s)$ representing expected team return
- Training: Uses global team reward signal (+50 for goals, -10 for losing)

This centralized training, decentralized execution (CTDE) architecture allows agents to coordinate during training while operating independently at inference time [7].

(5) Counterfactual Value Functions

MA-POCA computes each agent’s contribution using counterfactual baselines [7]. This is how it works in our case:

For agent i on the team, the team value estimations are:

$$V_{team}(s) = \text{Value if all agents act with the current } \pi$$

$$V_{team}^{-i}(s) = \text{Value if agent } i \text{ is replaced by baseline } \pi$$

Then, agent i ’s advantage is:

$$A_i = V_{team}(s) - V_{team}^{-i}(s)$$

This mechanism fairly distributes credit: agents who directly contributed to the goal receive stronger learning signals, while those uninvolved get proportionally smaller updates.

(6) Policy Updates

MA-POCA extends PPO’s clipped objective with counterfactual advantages (POCA = PPO + Counterfactuals):

$$\text{ratio} = \frac{\pi_{new}(\text{action}|\text{state})}{\pi_{old}(\text{action}|\text{state})}$$

$$\text{clipped_ratio} = \text{clip}(\text{ratio}, 1 - \epsilon, 1 + \epsilon)$$

$$L_{POCA} = \min(\text{ratio} * A_{cfact}, \text{clipped_ratio} * A_{cfact})$$

Hence, the same policy clipping mechanism from Stage 1 is applied (with the same $\epsilon=0.2$), while reflecting each agent’s individual contributions to team success rather than just local rewards.

(7) Hyperparameter Adjustments for Stage 2

Stage 2 modifies several hyperparameters from Stage 1 to accommodate multi-agent dynamics:

Table 3 - Modified Hyperparameters Between Stage 1 and 2

Parameter	Stage 1	Stage 2	Rationale
batch_size	8192	2048	Smaller batch per agent (4 agents share buffer)
buffer_size	81920	20480	Distributed across team (5120 steps/agent)
λ	0.0003	0.0001	Lower rate for fine-tuning transferred locomotion to prevent overwrite
γ	0.99	0.995	Higher discount for longer soccer sequences

More specifically, the higher γ accounts for longer causal chains in soccer, while the lower learning rate prevents forgetting of Stage 1’s locomotion skills.

(8) Self-Play and Competitive Training

Stage 2 incorporates self-play with ELO-based opponent selection, where each team plays against snapshots of past policies. Although the package provides an implementation of self-play, we had to modify it to prevent early policy collapse from competitive pressure through. Hence, we use a curriculum-controlled reward scaling, through the *self_play_weight* variable, to gradually encourage competition between the teams.

As such, in early training, scoring a goal awards value to the scoring team, while losing does not involve any sort of penalty.

Late in training (after hitting 50% of total steps), scoring still rewards value, but losing inquires a heavy -10 x *self_play_weight* penalty.

(9) Architecture Preservation for Transfer Learnings

Stage 2 initializes policy weights from the Stage 1 checkpoint, preserving all 269-dimensional input mappings.

In addition to the 243 locomotion features, the previously zeroed 26 soccer features are enabled at this stage.

(10) Reward and Penalty Structure

The Stage 2 reward structure is built upon the structure from Stage 1 and is designed to shape realistic, purposeful, and strategic soccer behaviors. It rewards positive ball influence, good field positioning, effective defense, and stable posture, while penalizing exploits, bunching, unsafe goalie behavior, and chaotic movement.

Here, the value estimation of each state is a mix of sums and products of the elements found in Table 7 from Appendix C.

(11) Training Configuration

The complete parameters for our training configuration can be found in the “WalkerSoccerStage2_Soccer.yaml” file.

5) RESULTS

As our walker robot uses a custom setup, there are no directly comparable models for us to compare to.

Nonetheless, the ML-Agents toolkit provides easy access to a plethora of metrics through a per run TensorBoard [8], which were instrumental to measuring the performance of the agent and adjusting hyperparameters throughout training.

Likewise, watching the agent at different intervals in the training was also very helpful for tuning the training parameters.

As the stages were inherently different, each stage’s performance was measured independently with the results of the best model for each stage presented below.

(1) Stage 1 performance

In Stage 1, the performance of the agent was mainly measured using the cumulative mean reward.

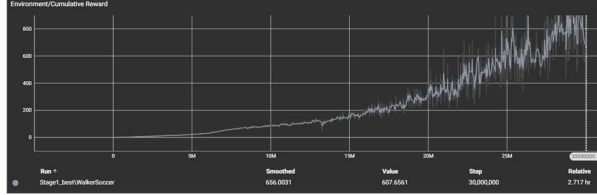


Figure 3 - Stage 1 best model cumulative mean reward

As seen from the graph in Figure 3, our final model continuously improved throughout training, reaching a final reward of about 608 and a maximal reward of 1041, and took more than 2.7 hours to train¹.

Observing the agent in action throughout training and comparing it to the graph, we noticed that the agent starts to successfully stand around 6.5M steps, after which it starts making its first steps and successfully touching a target until around 13M, when it starts touching multiple targets per episode and rewards start to vary mostly depending on how “lucky” the agent gets with the target spawns.

Looking at footage of the final agent in action, the agent’s walking is not completely perfect, as it still falls if it walks and turns too fast, but is more than good enough to work as a base for Stage 2.

(2) Stage 2 performance

In Stage 2, the performance of the agent was partially measured using the cumulative mean reward, but mostly through footage observation and ELO rating.

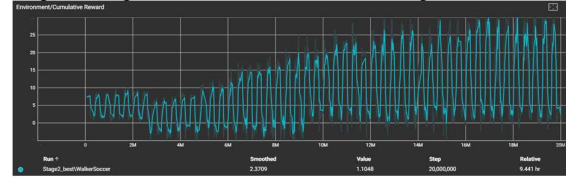


Figure 4 - Stage 2 best model cumulative mean reward

As seen from the graph in Figure 4, the mean cumulative reward exhibits three distinguishable phases: 0-5M steps (Lessons 1), 5M-10M steps (Lesson 2 transition), and 10M-20M steps (Lessons 3). Each phase corresponds to distinct curriculum configurations, with sharp transitions marking lesson boundaries where environmental parameters and reward structures change simultaneously. The agent requires an adaptation period following each transition, during which mean reward temporarily drops before recovering as the policy adjusts to the new task difficulty.

The consistent oscillations visible across all phases are primarily driven by self-play dynamics rather than learning instability. Self-play is active from step 0, causing agents to compete against policy snapshots saved every 200,000 steps. This creates a characteristic “rock-paper-scissors” pattern:

1. Exploitation phase (steps 0-200K): Agent discovers a strategy that beats the current opponent pool → reward increases
2. Snapshot incorporation (step 200K): New snapshot saves the successful strategy as a future opponent
3. Adversarial pressure (steps 200K-400K): Agent now faces its past successful self → temporary reward decrease while adapting
4. Counter-strategy emergence (steps 400K+): Agent evolves new tactics to beat previous strategies → reward recovers

This cycle repeats throughout training, with oscillation amplitude increasing over time as policies become more specialized and matchup-dependent. The oscillations are a healthy indicator of continuous policy improvement through adversarial competition, not a sign of training instability.

Overall, we noticed that the agent’s cumulative mean reward increased by the end of each phase, showing a first level of improvement.

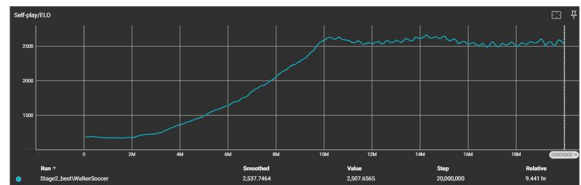


Figure 5 - Stage 2 ELO evolution

Examining the ELO rating graph in Figure 5 provides complementary insights into policy quality beyond raw cumulative reward. The ELO curve exhibits three distinct behavioral regimes that align with the curriculum phases identified in the reward analysis, but with important differences in interpretation.

¹Training times in this report were taken from a system using a i5-13600k CPU and RTX 4070 GPU and varied

from version to version. The longest training run took 20h for 50M steps in Stage 2 but yielded bad results

In Lessons 1 and 2 when competitive self-play is disabled, the ELO continuously increases showing that the agents are strictly getting better at scoring regardless of the opponent strength. At the end of Lesson 2, the ELO is 2665.

In Lesson 3 when penalties and competitive play are enabled, small amplitude oscillations appear. These oscillations are indicative of the shift of training objectives from “maximize goals scored” to “maximize goal differential”.

Hence, in Lesson 3, ELO becomes a measure of competitive strength rather than absolute skill.

At this point, the oscillations indicate another “rock-paper-scissors” cycle, where a strategy beating snapshot A might be losing B which leads to a non-transitive dominance hierarchy where no agent is definitively the best.

As the agent learns to trade off exploration of new strategies versus exploitation of known strategies, the peak ELO achieved is 2670 at 14.3M steps and is the model presented in the final test scene.

Observing the 14.3M model in action, we can see that the model has learnt to correctly shoot, pass and defend, showing a clear overall understanding of the game.

However, we quickly noticed that the goalie is not strictly following its assigned role, indicative that the agent is favoring taking goalie penalties to be more aggressive and offset the penalties through goals.

(3) Observed Behaviors and Strategies

Throughout our experiments, we noticed a multitude of interesting behaviors and strategies that boil down to the appearance of “reward hacking”.

In each stage, the agent’s goal is simple, maximize rewards. However, this makes it learn behaviors that maximize rewards rather than actually learn the correct behavior and required us to tediously modify the reward structures to combat these exploits and led to our final reward and penalty structures in Stage 1 and 2.

A breakout of the most interesting behaviors and strategies we observed is found in Appendix E.

6) DISCUSSION AND FUTURE WORK

The results of this project highlight both the strengths and limitations of applying reinforcement learning to high-dimensional humanoid agents in a competitive multi-agent environment.

The two-stage curriculum proved essential: agents trained with PPO in the locomotion phase developed stable walking and running behaviors, which transferred effectively into the soccer phase. In the multi-agent setting, MA-POCA and self-play enabled the agents to improve collaboratively despite sparse rewards and adversarial dynamics. The agents demonstrated clear progress in chasing the ball, executing kicks, and occasionally coordinating opportunistic scoring behaviors.

At the same time, their performance exposed challenges inherent to this domain. Limited spatial awareness led to suboptimal positioning, and the agents frequently discovered reward exploits such as corner-camping or diving to push the ball, underscoring the difficulty of crafting reward functions that guide learning without introducing unintended incentives.

These observations suggest several directions for future improvement.

Expanding the observation space to include goal positions, teammate locations, and opponent states would give agents richer situational understanding and reduce reliance on emergent behaviors driven solely by ball proximity. Introducing role-specific training, for example, separate policies for goalkeepers, defenders, and strikers, which could promote more structured teamwork and mitigate the issues caused by homogeneous policies.

The use of visual observations represents another promising avenue, allowing the agents to “see” the field and develop more natural spatial strategies, though this would require more computational resources and careful network design.

Additional refinements such as improved anti-exploit constraints, more expressive shaping rewards, or training soccer sub-skills with PPO before multi-agent integration may further enhance stability and performance. Together, these improvements could help bridge the gap between competent humanoid locomotion and coordinated, strategy-driven multi-agent soccer gameplay.

REFERENCES

- [1] “ML-Agents Overview | ML Agents | 4.0.0.” Accessed: Dec. 01, 2025. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.ml-agents@4.0/manual/index.html>
- [3] “ML-Agents Theory | ML Agents | 4.0.0.” Accessed: Dec. 01, 2025. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.ml-agents@4.0/manual/ML-Agents-Overview.html>
- [4] “Example Learning Environments | ML Agents | 4.0.0.” Accessed: Dec. 01, 2025. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.ml-agents@4.0/manual/Learning-Environment-Examples.html>
- [5] “Agents | ML Agents | 4.0.0.” Accessed: Dec. 02, 2025. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.ml-agents@4.0/manual/Learning-Environment-Design-Agents.html>
- [6] “ML-Agents Theory | ML Agents | 4.0.0.” Accessed: Dec. 02, 2025. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.ml-agents@4.0/manual/ML-Agents-Overview.html#training-in-cooperative-multi-agent-environments-with-ma-poca>
- [7] A. Cohen *et al.*, “On the Use and Misuse of Absorbing States in Multi-agent Reinforcement Learning,” June 07, 2022, *arXiv*: arXiv:2111.05992. doi: <https://arxiv.org/abs/2111.05992>.
- [8] “Using TensorBoard to Observe Training | ML Agents | 4.0.0.” Accessed: Dec. 02, 2025. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.ml-agents@4.0/manual/Using-Tensorboard.html>

APPENDIX

A. Comparison of Observation Methods

Table 4 - Pros and Cons of Vector Observation for Training Humanoid Soccer Agent

	Vector Observation
Pros	- Extremely fast to train due to low-dimensional input - Highly precise for proprioception - Stable and consistent across episodes; no noise from sensor discretization
Cons	- Limited perception range without explicit positional encodings - Requires manual design of all relevant features - Less flexible than visual inputs for arbitrary scene understanding

Table 5 - Pros and Cons of Visual Observation for Training Humanoid Soccer Agent

	Visual Observation
Pros	- Rich, generalizable perception capable of understanding complex scenes - More realistic for real-world soccer-like tasks - Allows implicit representation of the environment
Cons	- Training time is dramatically longer due to convolutional processing - Large policies often require GPUs and long wall-clock times - Walkers cannot visually observe their own limbs sufficiently for balance control

Table 6 - Pros and Cons of Raycast Observation for Training Humanoid Soccer Agent

	Raycast Observation
Pros	- Lower-dimensional than vision while still providing spatial sensing - Good for detecting obstacles, distances, or field boundaries - More structured than raw pixels
Cons	- Still slower to train than pure vector observations - Requires careful ray placement and tuning - Limited by ray resolution and angle, walkers cannot use raycasts to track their own limbs

B. Full Breakdown of 269 Observation Vector

- Locomotion Core (243):
 - Body part positions (relative): 16×3
 - Body part linear velocities: 16×3
 - Body part angular velocities: 16×3
 - Body part local rotations (quaternions, subset excluding hips/hands): 13×4
 - Joint strength ratios: 13×1
 - Velocity goal vs current speed distance: 1
 - Average velocity (relative): 3
 - Velocity goal (relative): 3
 - Rotation delta hips forward: 4 (quaternion)
 - Rotation delta head forward: 4 (quaternion)
 - Target position (Stage 1 only / zero placeholder Stage 2): 3
- Soccer Core (26):

- Ball position (3), Ball velocity (3)
- Team ID (1): Blue = +1, Purple = -1
- Role (1): Striker = +1, Goalie = -1, Generic = 0
- Own goal position (3), Opponent goal position (3)
- Teammate 1 position (3), Teammate 2 position (3)
- Opponent 1 position (3), Opponent 2 position (3)
- Stage 1: All 26 zeros (transfer consistency)
- Stage 2: Real data for role/goal/team spatial reasoning

C. Full Breakdown of 40 Continuous Action Vector

- Joint rotations (26): chest(3), spine(3), head(2), thighs(4), shins(2), feet(6), arms(4), forearms(2)
- Joint strengths (13): distributed across major appendages for dynamic stiffness control
- Kick intensity (1): enabled only when $\text{ball_touch} \geq 0.5$ (Lesson 3+)

D. Stage 2 Reward and Penalty Structure Table

Table 7 - Stage 2 Reward and Penalty Structure

Component	Value(s)	Purpose
Valid touch	+0.2	Reinforces upright, controlled gait rather than diving
Dive penalties	-0.15 to -1.35	Suppresses exploit of falling toward target for reset cycles
Turn encouragement	$+0.05 \times \text{progress}$	Ensures omnidirectional capability (critical for soccer pursuit)
Standing penalty	$-0.02 \times \text{angle frac}$	Discourages inertia lock when target moves laterally
Upright bonus	+0.03	Maintains stable posture in chaotic scrums
Goals (score)	$+50 \times \text{time_bonus}$	Dominant sparse objective for team success
Goals (concede)	$-10 \times \text{self_play_weight}$	Scales competitive pressure safely via curriculum
Ball touches	$+0.2 \times \text{curriculum}$	Progressively increases salience of ball interaction
Kick bonus	+0.1	Encourages deliberate kicks post-Lesson 2
Role (Striker)	+0.015 near opponent goal	Encourages forward offensive positioning
Role (Goalie)	+0.015 alignment ball-goal	Reinforces blocking alignment & anchoring
Own-goal velocity	$-0.04 \times \text{toward speed}$	Strong deterrent to accidental clears backward
Own-goal proximity	$-0.08 \times \text{proximity factor}$	Increased penalty near danger zone
Clearance progress	+0.01 moving ball out	Positive shaping for defensive recoveries

Spacing penalty	-0.005 per close teammate	Mitigates bunching around ball
Marking bonus	+0.005 per opponent within 3m	Promotes zone control & defense
Corner camping	-0.08	Removes exploit of respawn-center tactic
Ball freed from corner	+0.02	Encourages active repositioning
Goalie leash penalty	Dynamic -0.006/m outside leash	Keeps goalie in defendable zone
Goalie block bonus	+0.03-0.07	Rewards accurate alignment with incoming ball

E. Breakdown of Observed Behaviors and Strategies

Stage (S) / Lesson (L)	Behavior/Rationale
S1	Run and dive to touch target faster. Uses stand respawn to maximize targets touched per episode
S1	If target respawned behind or generally >30° from current direction, fall and reset since it is faster than slowing down and turning
S2L1	Get ball stuck in corner or on field edge. Allows ball_touch farming
S2L2	Self-discovered physics-based kicking (using leg rather than kick action). As action-based kicks had a penalty for missing, physics-based kicks are free (but not as powerful)
S2L3	Bunch and pile up to create “pseudo wall”. Ball isn’t small enough to pass through the model’s legs making it very hard to let the ball through and significantly reduces losing matches

F. Disclaimer About the Use of AI

For this project, AI was used in the following ways:

- Help research the choice of algorithms such as PPO and MA-POCA
- Help implement our custom observation and action vectors
- Help implement our custom reward structures and other Unity related mechanics
- Help tune hyperparameters, environmental parameters and reward/penalty structure values to encourage agent behaviors in stage 1 and 2
- Suggest ideas on how to speed up training
- Generate summary documents to help prepare the presentation and final report
- Enhance flow and wording within the final report

Note: All specific ideas for the implementation ideas were provided by us or suggested in documentation we found with AI being used to figure out to implement the idea in Unity C# code (which we were less familiar with)